МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «Информационных технологий»

Лабораторная работа № 6
по дисциплине «Введение в информационные технологии»
по теме: «**РАЗРАБОТКА GUI-ПРИЛОЖЕНИЙ НА PYQT6**»

Выполнил студент группы:
БВТ2601
Гутаев Эльдар Назирович
Проверил:

Москва 2026

Оглавление

Введение……3

Выполнение 4

Вывод..... 28

Введение

Цель и задачи работы:

Целью данной лабораторной работы является: изучить основные принципы разработки графических пользовательских интерфейсов средствами PyQt6, научиться создавать окна приложения, размещать и настраивать виджеты, использовать менеджеры компоновки, обрабатывать действия пользователя с помощью механизма сигналов и слотов и разрабатывать простое настольное приложение.

Задачи: В ходе выполнения лабораторной работы необходимо создать законченное графическое приложение «Менеджер задач»: последовательно пройти путь от пустого окна до приложения, которое добавляет, удаляет и учитывает задачи, обрабатывает некорректные действия пользователя, запрашивает подтверждение опасных операций и поддерживает согласованное состояние интерфейса.

Выполнение

Задание 1. Создан каталог проекта task_manager (в общей папке лабораторных работ он назван lab_06_task_manager) и открыт в Visual Studio Code. В редакторе VS Code у меня нет ввода с клавиатуры в терминал, поэтому команды оформлены как задачи VS Code (файл .vscode/tasks.json) и запущены через меню Terminal → Run Task; вывод в терминале — результат реального выполнения команд. На рисунке 1: командой git init создан пустой репозиторий, python -m venv .venv создал виртуальное окружение, в него установлен PyQt6 (PyQt6 6.12.0, PyQt6-Qt6 6.12.0, PyQt6_sip 13.13.0), командой pip freeze сформирован requirements.txt, содержимое которого выведено в конце. Вместо активации через Activate.ps1 (её блокирует политика выполнения PowerShell) pip вызывался интерпретатором .venv\Scripts\python напрямую — пакеты ставятся в то же окружение. В .gitignore добавлены .venv/, __pycache__/, *.pyc и служебный .vscode/. На рисунке 1.1: первый коммит f9db2d6 «Начальная структура проекта

task_manager» включил .gitignore и requirements.txt, git status показывает чистое рабочее дерево; каталог .venv в коммит не попал.

```
git init
python -m venv .venv
.venv\Scripts\python -m pip install PyQt6
.venv\Scripts\python -m pip freeze > requirements.txt
git add .
git commit -m "Начальная структура проекта task_manager"
git log --oneline
git status

.venv/
__pycache__/
*.pyc
.vscode/
```

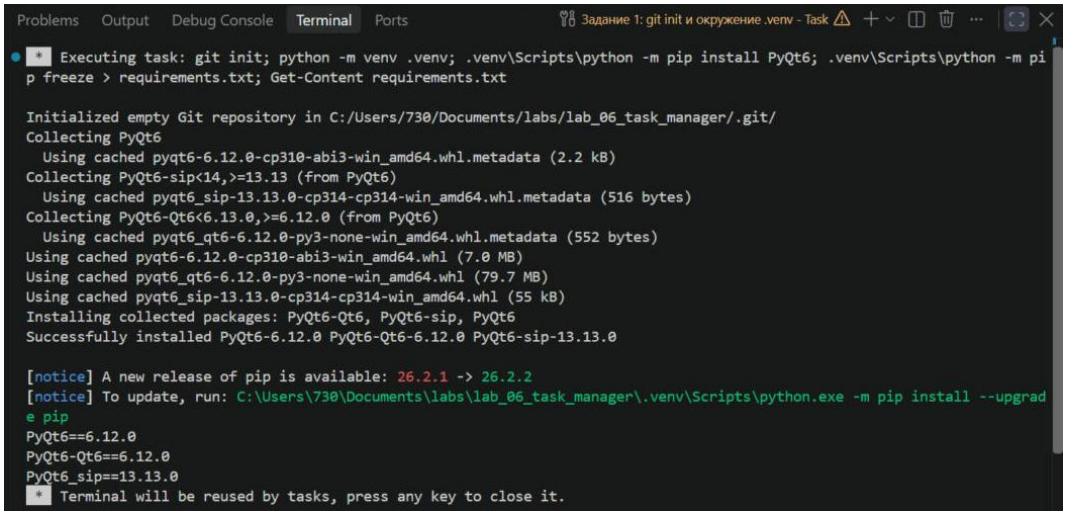


Рисунок 1. Репозиторий, окружение .venv и установка PyQt6

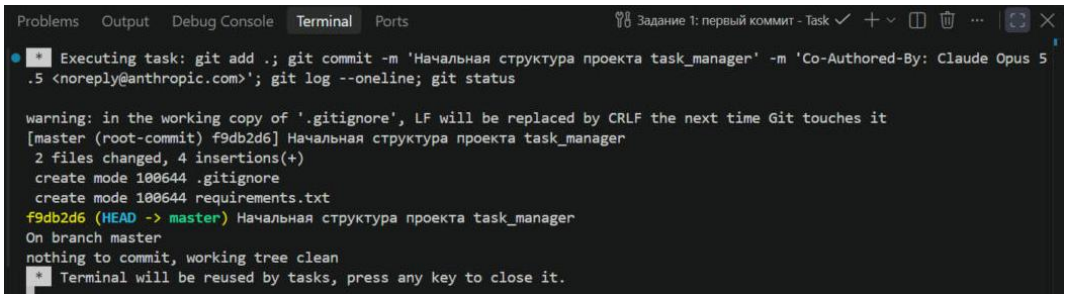


Рисунок 1.1. Первый коммит проекта

Задание 2. Созданы main.py и main_window.py. В main.py функция main() создаёт QApplication, окно MainWindow, показывает его методом show() и запускает цикл обработки событий app.exec(); пока цикл работает, программа не завершается и ждёт действий пользователя. Класс MainWindow наследует QMainWindow, задаёт заголовок «Менеджер задач» и размер 500 × 400. На рисунке 2: после запуска кнопкой Run открылось пустое окно с нужным

заголовком. В терминале видно, что VS Code запустил программу интерпретатором из .venv.

```
import sys

from PyQt6.QtWidgets import QApplication

from main_window import MainWindow


def main():
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())


if __name__ == "__main__":
    main()
```

```
from PyQt6.QtWidgets import QMainWindow


class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Менеджер задач")
        self.resize(500, 400)
```

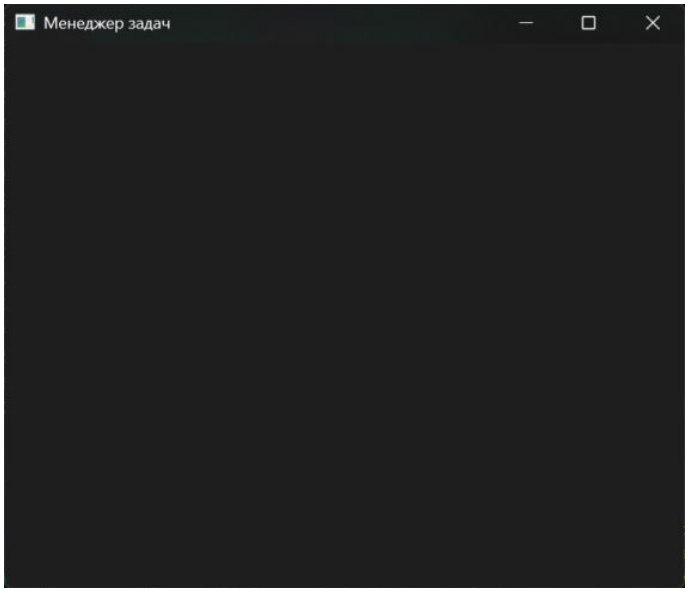


Рисунок 2. Пустое главное окно

Задание 3. На рисунке 3: в окно добавлены QLabel «Новая задача», QLineEdit с подсказкой «Название задачи», кнопка «Добавить», QListWidget, кнопки «Удалить» и «Очистить всё» и QLabel со счётчиком. Чтобы QMainWindow показал элементы, они временно помещены в один вертикальный

QVBoxLayout на центральном виджете — без этого виджеты наложились бы друг на друга в левом верхнем углу. Обработчиков пока нет, элементы только отображаются.

```
from PyQt6.QtWidgets import (
    QLabel,
    QLineEdit,
    QListWidget,
    QMainWindow,
    QPushButton,
    QVBoxLayout,
    QWidget,
)


class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Менеджер задач")
        self.resize(500, 400)

        self.title_label = QLabel("Новая задача")
        self.task_input = QLineEdit()
        self.task_input.setPlaceholderText("Название задачи")
        self.add_button = QPushButton("Добавить")
        self.task_list = QListWidget()
        self.delete_button = QPushButton("Удалить")
        self.clear_button = QPushButton("Очистить всё")
        self.counter_label = QLabel("Количество задач: 0")

        # Пока элементы просто выводятся друг под другом,
        # компоновка по строкам выполняется в задании 4.
        central_widget = QWidget()
        layout = QVBoxLayout(central_widget)
        for widget in (
            self.title_label,
            self.task_input,
            self.add_button,
            self.task_list,
            self.delete_button,
            self.clear_button,
            self.counter_label,
        ):
            layout.addWidget(widget)
        self.setCentralWidget(central_widget)
```

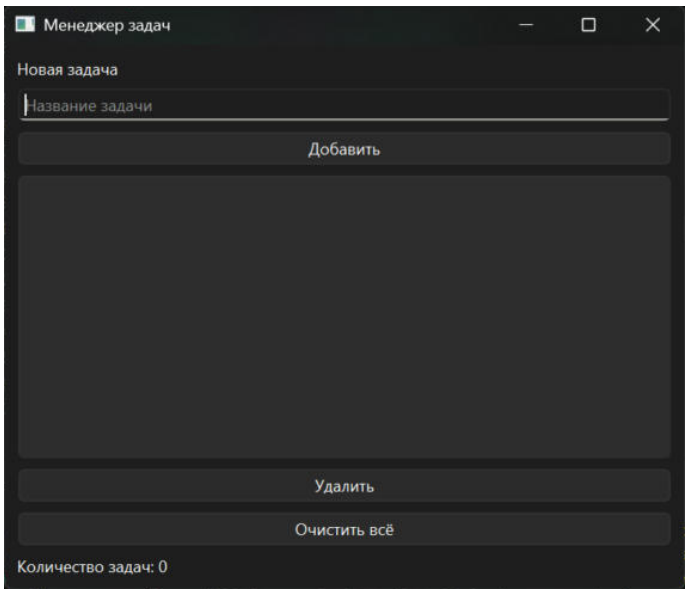


Рисунок 3. Элементы интерфейса

Задание 4. Создан центральный QWidget, установленный методом setCentralWidget(); основная компоновка — QVBoxLayout. Поле ввода и кнопка «Добавить» помещены в один QHBoxLayout, кнопки «Удалить» и «Очистить всё» — в другой, оба вложены в основной layout через addLayout(). Абсолютные координаты и move() не используются. На рисунке 4 — окно обычного размера, на рисунке 4.1 — развёрнутое на весь экран: поле ввода растянулось на всю ширину, а кнопка «Добавить» сохранила свой

размер, список занял всю свободную высоту, кнопки нижней строки поделили ширину поровну. Компоновка пересчитала размеры сама.

```
from PyQt6.QtWidgets import (
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QListWidget,
    QMainWindow,
    QPushButton,
    QVBoxLayout,
    QWidget,
)


class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Менеджер задач")
        self.resize(500, 400)

        self.title_label = QLabel("Новая задача")
        self.task_input = QLineEdit()
        self.task_input.setPlaceholderText("Название задачи")
        self.add_button = QPushButton("Добавить")
        self.task_list = QListWidget()
        self.delete_button = QPushButton("Удалить")
        self.clear_button = QPushButton("Очистить всё")
        self.counter_label = QLabel("Количество задач: 0")

        input_layout = QHBoxLayout()
        input_layout.addWidget(self.task_input)
        input_layout.addWidget(self.add_button)

        buttons_layout = QHBoxLayout()
        buttons_layout.addWidget(self.delete_button)
        buttons_layout.addWidget(self.clear_button)

        main_layout = QVBoxLayout()
        main_layout.addWidget(self.title_label)
        main_layout.addLayout(input_layout)
        main_layout.addWidget(self.task_list)
        main_layout.addLayout(buttons_layout)
        main_layout.addWidget(self.counter_label)

        central_widget = QWidget()
        central_widget.setLayout(main_layout)
        self.setCentralWidget(central_widget)
```

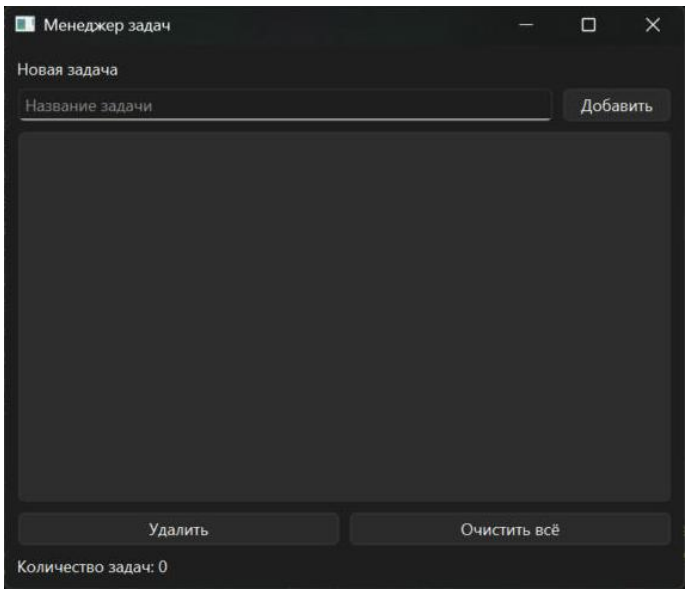


Рисунок 4. Окно с вложенными компоновками



Рисунок 4.1. Компоновка при растягивании окна

Задание 5. Создан метод add_task(), сигнал clicked кнопки «Добавить» связан с ним методом connect(). Метод берёт текст из QLineEdit, удаляет пробелы по краям методом strip(), и если строка не пустая — добавляет её в QListWidget и очищает поле. На рисунке 5: первая задача «Купить молоко» уже в списке, в поле набрана вторая. На рисунке 5.1: после нажатия «Добавить» вторая задача появилась в списке, а попытка добавить строку из трёх пробелов

ничего не добавила — после strip() она пустая. Счётчик внизу пока не меняется: его обновление реализуется в задании 11.

```
        self.add_button.clicked.connect(self.add_task)

    def add_task(self):
        text = self.task_input.text().strip()
        if text:
            self.task_list.addItem(text)
            self.task_input.clear()
```

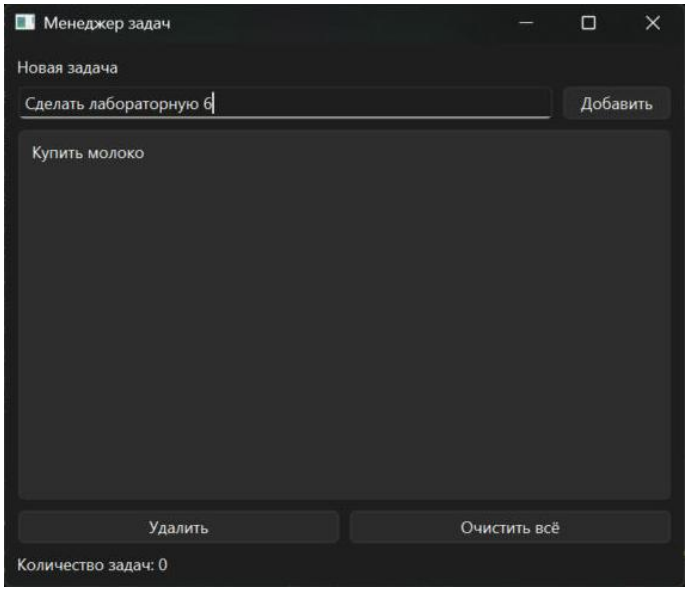


Рисунок 5. Ввод второй задачи

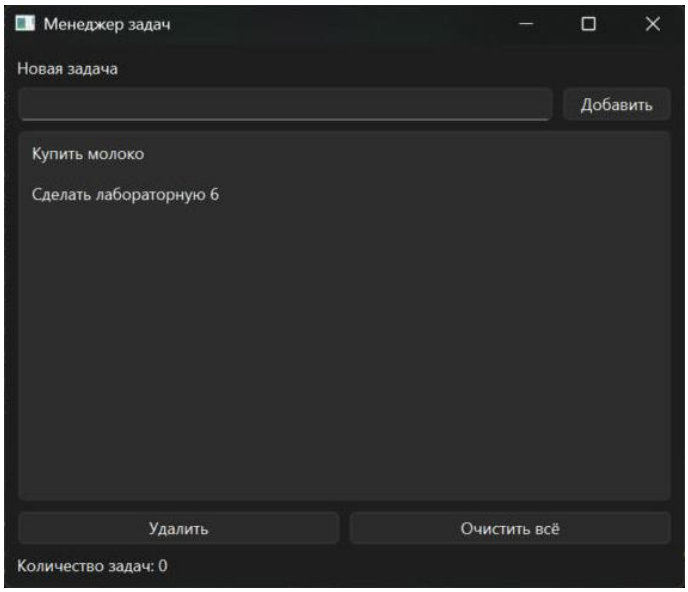


Рисунок 5.1. Задачи добавлены кнопкой «Добавить»

Задание 6. Метод add_task() дополнен проверкой: если после strip() строка пустая, выводится QMessageBox.warning и метод завершается через return, не добавляя задачу. На рисунке 6: в поле введены только пробелы, после

нажатия «Добавить» появилось окно «Ошибка» с текстом «Введите название задачи», список остался пустым.

```
    def add_task(self):
        text = self.task_input.text().strip()
        if not text:
            QMessageBox.warning(self, "Ошибка", "Введите
название задачи")
            return
        self.task_list.addItem(text)
        self.task_input.clear()
```

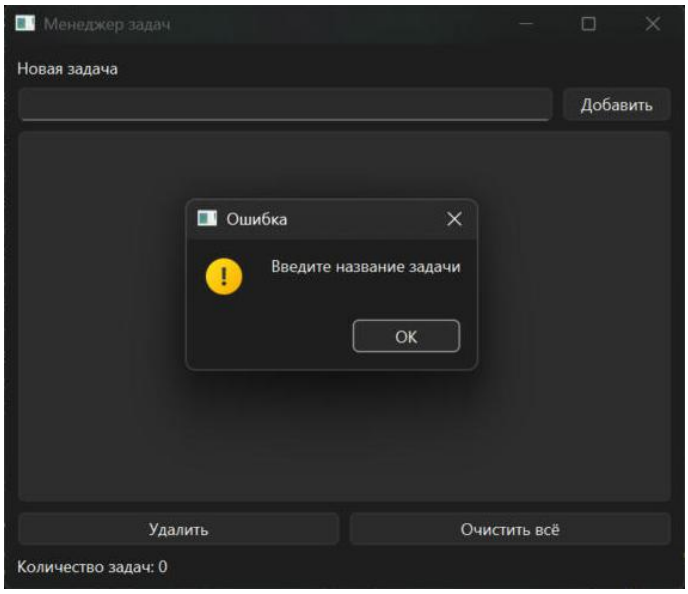


Рисунок 6. Предупреждение при пустом вводе

Задание 7. Сигнал returnPressed поля QLineEdit связан с тем же методом add_task(), отдельный обработчик не создавался. На рисунке 7: первые две

задачи добавлены нажатием Enter, третья — кнопкой «Добавить». Результат одинаковый, потому что оба сигнала вызывают один и тот же слот.

```
self.add_button.clicked.connect(self.add_task)
self.task_input.returnPressed.connect(self.add_task)
```

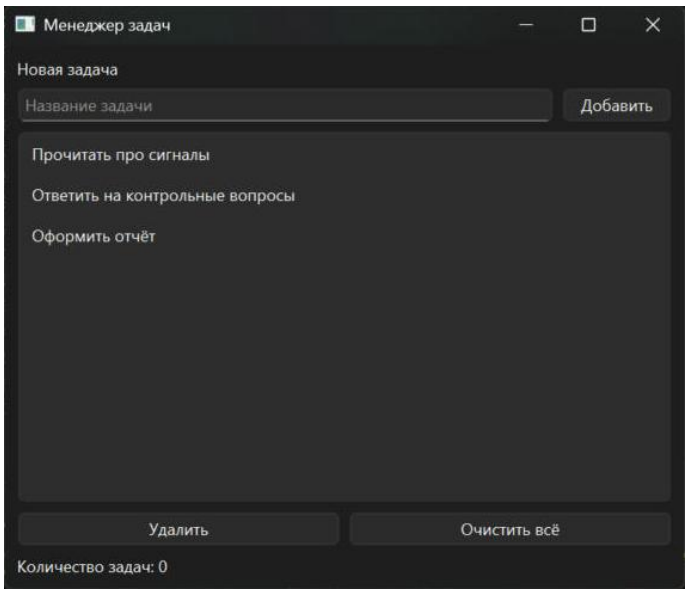


Рисунок 7. Добавление задач клавишей Enter и кнопкой

Задание 8. Создан метод delete_task(), связанный с кнопкой «Удалить». Он получает выбранную строку методом currentRow(); если ничего не выбрано, метод возвращает −1, и выводится предупреждение. На рисунке 8: кнопка нажата без выбора — появилось сообщение «Выберите задачу для

удаления». На рисунке 8.1: выбрана задача «Сдать лабораторную 5», после нажатия «Удалить» она удалена методом takeItem(row).

```
        self.delete_button.clicked.connect(self.delete_task)

    def delete_task(self):
        row = self.task_list.currentRow()
        if row < 0:
            QMessageBox.warning(self, "Ошибка", "Выберите задачу
для удаления")
            return
        self.task_list.takeItem(row)
```

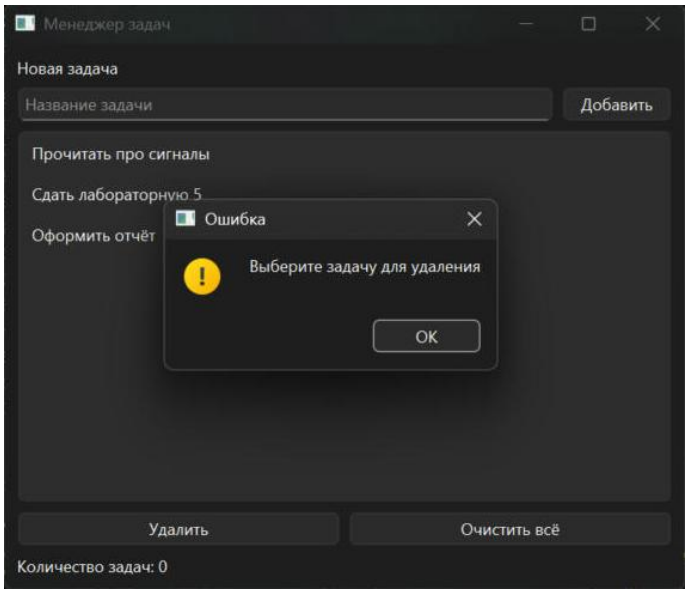


Рисунок 8. Удаление без выбранной задачи

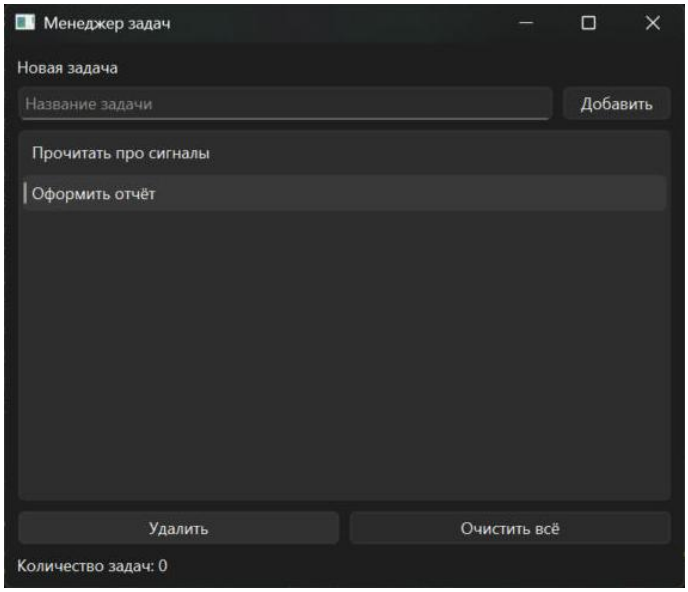


Рисунок 8.1. Выбранная задача удалена

Задание 9. Перед удалением выводится QMessageBox.question с названием выбранной задачи, и takeItem() выполняется только при ответе

StandardButton.Yes. На рисунке 9: окно «Удаление» с вопросом «Удалить задачу «Сдать лабораторную 5»?». При ответе No задача осталась в списке, при повторном нажатии и ответе Yes — удалена (рисунок 9.1). Подписи кнопок Yes/No английские, потому что переводы Qt в приложение не подключались.

```
    def delete_task(self):
        row = self.task_list.currentRow()
        if row < 0:
            QMessageBox.warning(self, "Ошибка", "Выберите задачу
для удаления")
            return
        task = self.task_list.item(row).text()
        answer = QMessageBox.question(self, "Удаление",
f"Удалить задачу «{task}»?")
        if answer == QMessageBox.StandardButton.Yes:
            self.task_list.takeItem(row)
```

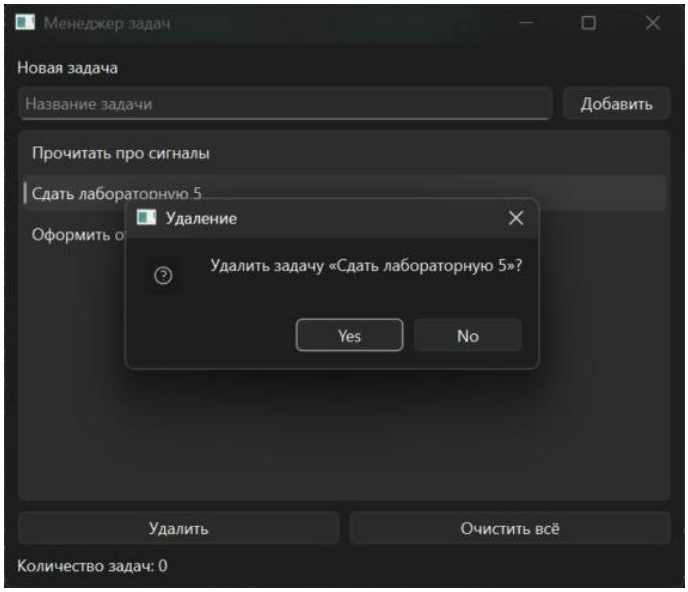


Рисунок 9. Запрос подтверждения удаления

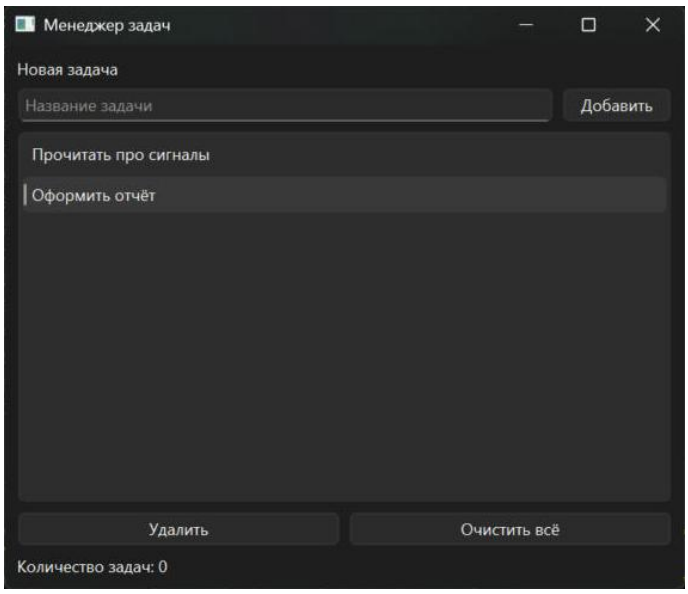


Рисунок 9.1. Задача удалена после ответа Yes

Задание 10. Создан метод clear_tasks(), вызываемый кнопкой «Очистить всё». Если список пуст, выводится информационное сообщение и метод завершается — на рисунке 10 кнопка нажата при пустом списке, показано «Список задач уже пуст», программа продолжает работу. Если задачи есть,

запрашивается подтверждение «Удалить все задачи?» (рисунок 10.1); после ответа Yes список очищен методом clear().

```
        self.clear_button.clicked.connect(self.clear_tasks)

    def clear_tasks(self):
        if self.task_list.count() == 0:
            QMessageBox.information(self, "Очистка", "Список
задач уже пуст")
            return
        answer = QMessageBox.question(self, "Очистка", "Удалить
все задачи?")
        if answer == QMessageBox.StandardButton.Yes:
            self.task_list.clear()
```

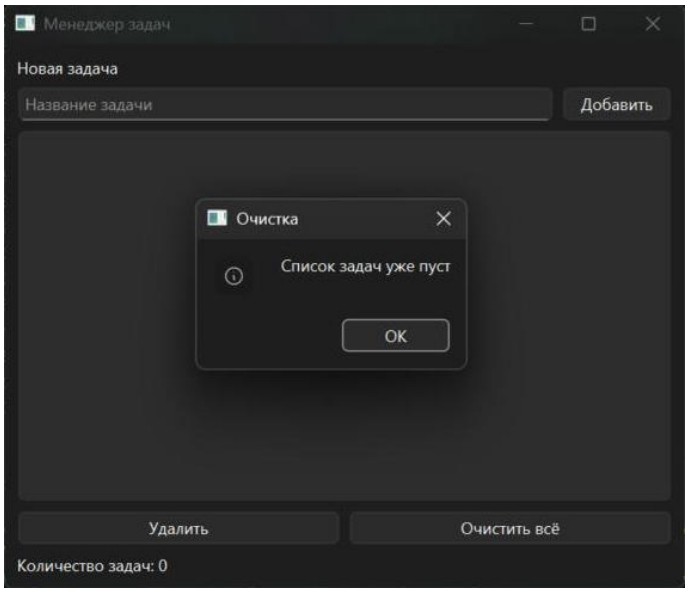


Рисунок 10. Очистка пустого списка

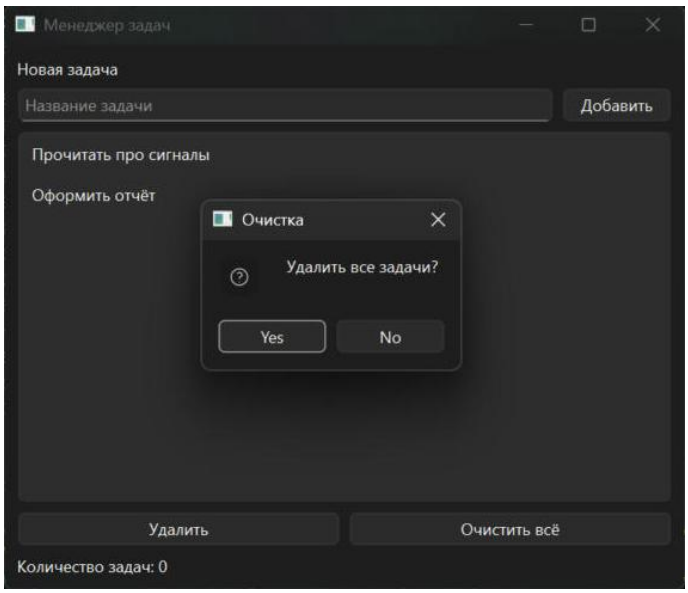


Рисунок 10.1. Подтверждение очистки списка

Задание 11. Создан метод update_counter(), который записывает в QLabel количество элементов QListWidget в формате «Количество задач: N». Он вызывается после успешного добавления, после удаления выбранной задачи и после очистки. На предыдущих рисунках счётчик оставался равным 0 при непустом списке — интерфейс не соответствовал данным. Теперь после добавления трёх задач показано «Количество задач: 3» (рисунок 11), после удаления одной — 2 (рисунок 11.1), после очистки — 0 (рисунок 11.2).

```
        def update_counter(self):
            count = self.task_list.count()
            self.counter_label.setText(f"Количество задач: {count}")

        def add_task(self):
            text = self.task_input.text().strip()
            if not text:
                QMessageBox.warning(self, "Ошибка", "Введите
    название задачи")
                return
            self.task_list.addItem(text)
            self.task_input.clear()
            self.update_counter()
```

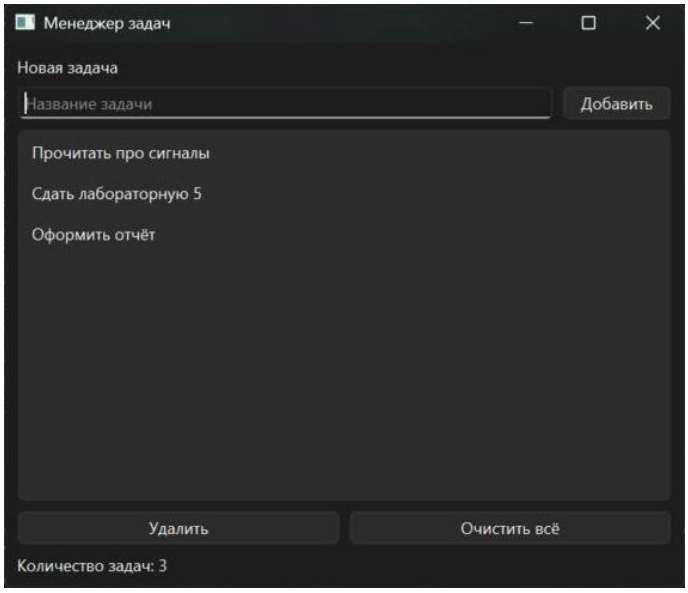


Рисунок 11. Счётчик после добавления трёх задач

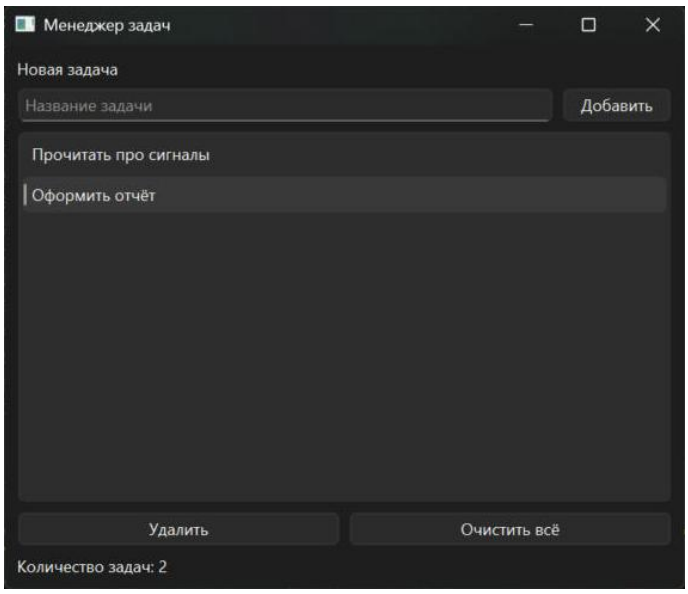


Рисунок 11.1. Счётчик после удаления задачи

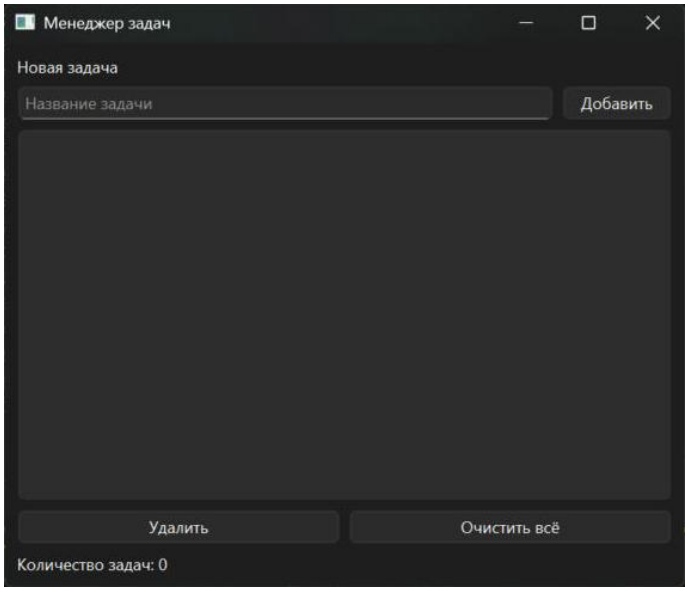


Рисунок 11.2. Счётчик после очистки списка

Задание 12. Класс MainWindow разделён на методы: init_ui() создаёт виджеты и компоновки, connect_signals() связывает сигналы со слотами, add_task(), delete_task(), clear_tasks() и update_counter() обрабатывают действия пользователя. Метод __init__() теперь только вызывает init_ui(), connect_signals() и update_counter(). Структура класса видна в панели Outline

на рисунке 12. На рисунке 12.1: после разделения приложение работает как раньше — задачи добавляются и Enter, и кнопкой, счётчик показывает 2.

```
from PyQt6.QtWidgets import (
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QListWidget,
    QMainWindow,
    QMessageBox,
    QPushButton,
    QVBoxLayout,
    QWidget,
)


class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.init_ui()
        self.connect_signals()
        self.update_counter()

    def init_ui(self):
        self.setWindowTitle("Менеджер задач")
        self.resize(500, 400)

        self.title_label = QLabel("Новая задача")
        self.task_input = QLineEdit()
        self.task_input.setPlaceholderText("Название задачи")
        self.add_button = QPushButton("Добавить")
        self.task_list = QListWidget()
        self.delete_button = QPushButton("Удалить")
        self.clear_button = QPushButton("Очистить всё")
        self.counter_label = QLabel()

        input_layout = QHBoxLayout()
        input_layout.addWidget(self.task_input)
        input_layout.addWidget(self.add_button)

        buttons_layout = QHBoxLayout()
        buttons_layout.addWidget(self.delete_button)
        buttons_layout.addWidget(self.clear_button)

        main_layout = QVBoxLayout()
        main_layout.addWidget(self.title_label)
        main_layout.addLayout(input_layout)
        main_layout.addWidget(self.task_list)
        main_layout.addLayout(buttons_layout)
        main_layout.addWidget(self.counter_label)

        central_widget = QWidget()
        central_widget.setLayout(main_layout)
        self.setCentralWidget(central_widget)

    def connect_signals(self):
```

```
        self.add_button.clicked.connect(self.add_task)
        self.task_input.returnPressed.connect(self.add_task)
        self.delete_button.clicked.connect(self.delete_task)
        self.clear_button.clicked.connect(self.clear_tasks)

    def add_task(self):
        text = self.task_input.text().strip()
        if not text:
            QMessageBox.warning(self, "Ошибка", "Введите
название задачи")
            return
        self.task_list.addItem(text)
        self.task_input.clear()
        self.update_counter()

    def delete_task(self):
        row = self.task_list.currentRow()
        if row < 0:
            QMessageBox.warning(self, "Ошибка", "Выберите задачу
для удаления")
            return
        task = self.task_list.item(row).text()
        answer = QMessageBox.question(self, "Удаление",
f"Удалить задачу «{task}»?")
        if answer == QMessageBox.StandardButton.Yes:
            self.task_list.takeItem(row)
            self.update_counter()

    def clear_tasks(self):
        if self.task_list.count() == 0:
            QMessageBox.information(self, "Очистка", "Список
задач уже пуст")
            return
        answer = QMessageBox.question(self, "Очистка", "Удалить
все задачи?")
        if answer == QMessageBox.StandardButton.Yes:
            self.task_list.clear()
            self.update_counter()

    def update_counter(self):
        count = self.task_list.count()
        self.counter_label.setText(f"Количество задач: {count}")
```

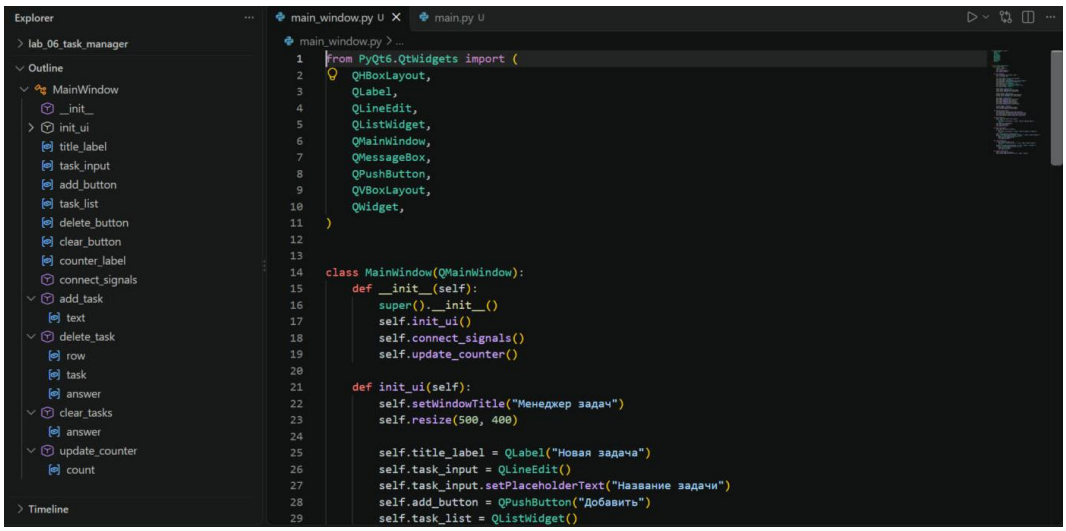


Рисунок 12. Методы класса MainWindow в панели Outline

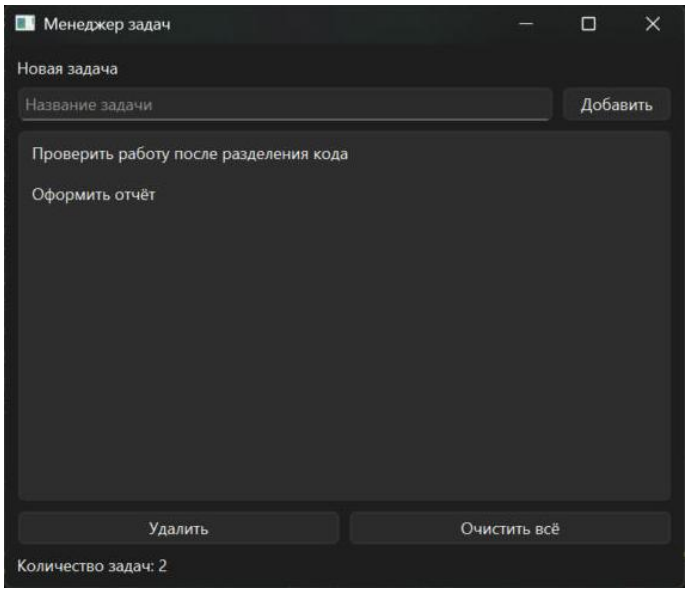


Рисунок 12.1. Работа приложения после разделения кода

Задание 13. Самостоятельно добавлено шесть возможностей: выбор приоритета задачи в QComboBox («Обычный», «Важный», «Срочный»; срочные задачи выделяются красным, важные — оранжевым), отметка выбранной задачи как выполненной кнопкой «Выполнено» (зачёркивание и серый цвет, повторное нажатие снимает отметку), удаление только выполненных задач, отображение количества выполненных в счётчике, количество задач в заголовке окна и сообщения в строке состояния QMainWindow. Состояние задачи хранится в самом элементе списка через setData() с ролями UserRole, поэтому отдельный список данных не нужен. На рисунке 13 добавлены четыре задачи с разными приоритетами, заголовок —

«Менеджер задач — задач: 4», в строке состояния — последняя добавленная задача. На рисунке 13.1 две задачи отмечены выполненными, счётчик — «Количество задач: 4 (выполнено: 2)». На рисунке 13.2 после «Удалить выполненные» остались две задачи, заголовок и счётчик обновились, в строке состояния — «Удалено выполненных задач: 2». Старые возможности

(Enter, предупреждения, подтверждения) продолжают работать, так как остались в тех же обработчиках.

```
from PyQt6.QtCore import Qt
from PyQt6.QtGui import QColor
from PyQt6.QtWidgets import (
    QComboBox,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QListWidget,
    QListWidgetItem,
    QMainWindow,
    QMessageBox,
    QPushButton,
    QVBoxLayout,
    QWidget,
)

DONE_ROLE = Qt.ItemDataRole.UserRole
PRIORITY_ROLE = Qt.ItemDataRole.UserRole + 1
PRIORITY_COLORS = {"Важный": "#e08a00", "Срочный": "#d93025"}
DONE_COLOR = "#8a8a8a"


class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.init_ui()
        self.connect_signals()
        self.update_counter()
        self.statusBar().showMessage("Готово к работе")

    def init_ui(self):
        self.resize(560, 420)

        self.title_label = QLabel("Новая задача")
        self.task_input = QLineEdit()
        self.task_input.setPlaceholderText("Название задачи")
        self.priority_box = QComboBox()
        self.priority_box.addItems(["Обычный", "Важный",
"Срочный"])
        self.add_button = QPushButton("Добавить")
        self.task_list = QListWidget()
        self.done_button = QPushButton("Выполнено")
        self.delete_button = QPushButton("Удалить")
        self.delete_done_button = QPushButton("Удалить
выполненные")
        self.clear_button = QPushButton("Очистить всё")
        self.counter_label = QLabel()

        input_layout = QHBoxLayout()
        input_layout.addWidget(self.task_input)
        input_layout.addWidget(self.priority_box)
        input_layout.addWidget(self.add_button)

        buttons_layout = QHBoxLayout()
```

```
        buttons_layout.addWidget(self.done_button)
        buttons_layout.addWidget(self.delete_button)
        buttons_layout.addWidget(self.delete_done_button)
        buttons_layout.addWidget(self.clear_button)

        main_layout = QVBoxLayout()
        main_layout.addWidget(self.title_label)
        main_layout.addLayout(input_layout)
        main_layout.addWidget(self.task_list)
        main_layout.addLayout(buttons_layout)
        main_layout.addWidget(self.counter_label)

        central_widget = QWidget()
        central_widget.setLayout(main_layout)
        self.setCentralWidget(central_widget)

    def connect_signals(self):
        self.add_button.clicked.connect(self.add_task)
        self.task_input.returnPressed.connect(self.add_task)
        self.done_button.clicked.connect(self.toggle_done)
        self.delete_button.clicked.connect(self.delete_task)

self.delete_done_button.clicked.connect(self.delete_done_tasks)
        self.clear_button.clicked.connect(self.clear_tasks)

    def add_task(self):
        text = self.task_input.text().strip()
        if not text:
            QMessageBox.warning(self, "Ошибка", "Введите
название задачи")
            return
        priority = self.priority_box.currentText()
        item = QListWidgetItem(f"[{priority}] {text}")
        item.setData(DONE_ROLE, False)
        item.setData(PRIORITY_ROLE, priority)
        self.paint_item(item)
        self.task_list.addItem(item)
        self.task_input.clear()
        self.update_counter()
        self.statusBar().showMessage(f"Добавлена задача:
{text}")

    def toggle_done(self):
        item = self.task_list.currentItem()
        if item is None:
            QMessageBox.warning(self, "Ошибка", "Выберите
задачу")
            return
        done = not item.data(DONE_ROLE)
        item.setData(DONE_ROLE, done)
        self.paint_item(item)
        self.update_counter()
        if done:
            self.statusBar().showMessage("Задача отмечена как
выполненная")
        else:
```

```
            self.statusBar().showMessage("Отметка о выполнении
снята")

    def paint_item(self, item):
        done = item.data(DONE_ROLE)
        font = item.font()
        font.setStrikeOut(done)
        item.setFont(font)
        color = DONE_COLOR if done else
PRIORITY_COLORS.get(item.data(PRIORITY_ROLE))
        item.setData(Qt.ItemDataRole.ForegroundRole,
QColor(color) if color else None)

    def delete_task(self):
        row = self.task_list.currentRow()
        if row < 0:
            QMessageBox.warning(self, "Ошибка", "Выберите задачу
для удаления")
            return
        task = self.task_list.item(row).text()
        answer = QMessageBox.question(self, "Удаление",
f"Удалить задачу «{task}»?")
        if answer == QMessageBox.StandardButton.Yes:
            self.task_list.takeItem(row)
            self.update_counter()
            self.statusBar().showMessage("Задача удалена")

    def delete_done_tasks(self):
        done_rows = [
            row
            for row in range(self.task_list.count())
            if self.task_list.item(row).data(DONE_ROLE)
        ]
        if not done_rows:
            QMessageBox.information(self, "Удаление",
"Выполненных задач нет")
            return
        for row in reversed(done_rows):
            self.task_list.takeItem(row)
        self.update_counter()
        self.statusBar().showMessage(f"Удалено выполненных
задач: {len(done_rows)}")

    def clear_tasks(self):
        if self.task_list.count() == 0:
            QMessageBox.information(self, "Очистка", "Список
задач уже пуст")
            return
        answer = QMessageBox.question(self, "Очистка", "Удалить
все задачи?")
        if answer == QMessageBox.StandardButton.Yes:
            self.task_list.clear()
            self.update_counter()
            self.statusBar().showMessage("Список задач очищен")

    def count_done(self):
        return sum(
```

```
                1
                for row in range(self.task_list.count())
                if self.task_list.item(row).data(DONE_ROLE)
            )

    def update_counter(self):
        count = self.task_list.count()
        done = self.count_done()
        self.counter_label.setText(f"Количество задач: {count}
(выполнено: {done})")
        self.setWindowTitle(f"Менеджер задач — задач: {count}")
```

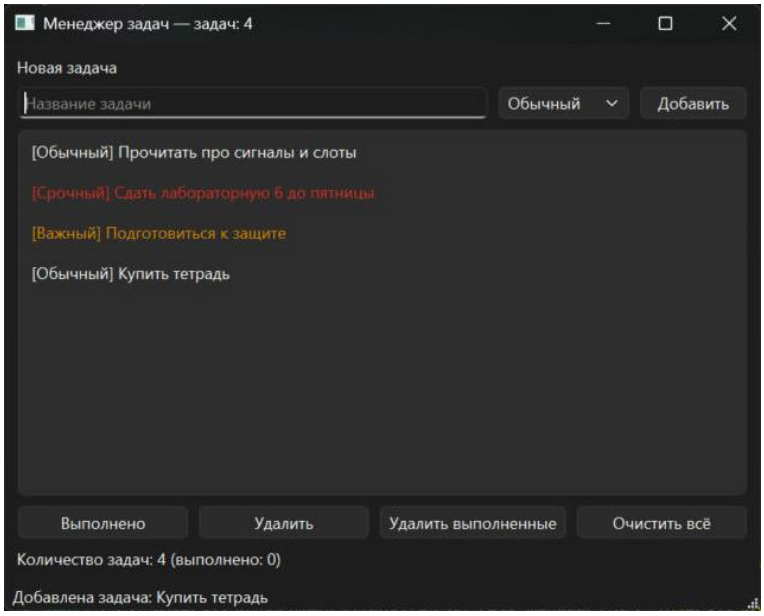


Рисунок 13. Задачи с приоритетами

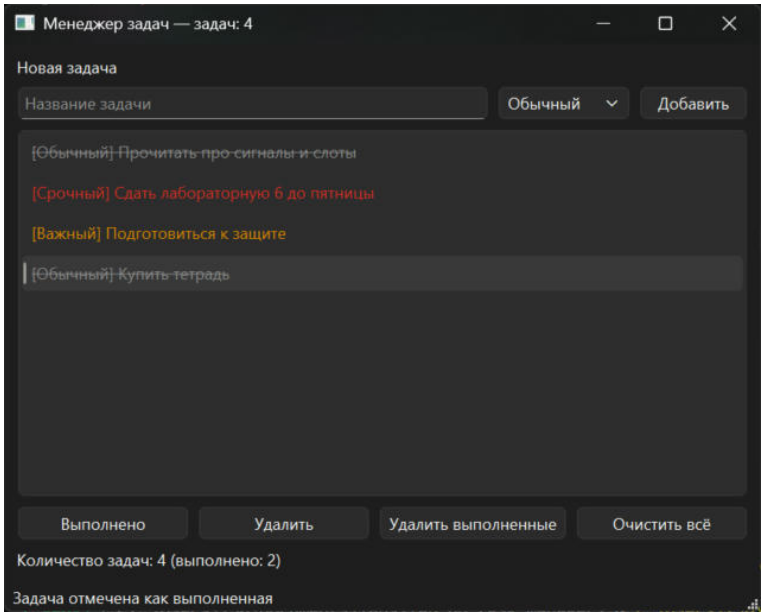


Рисунок 13.1. Отметка выполненных задач

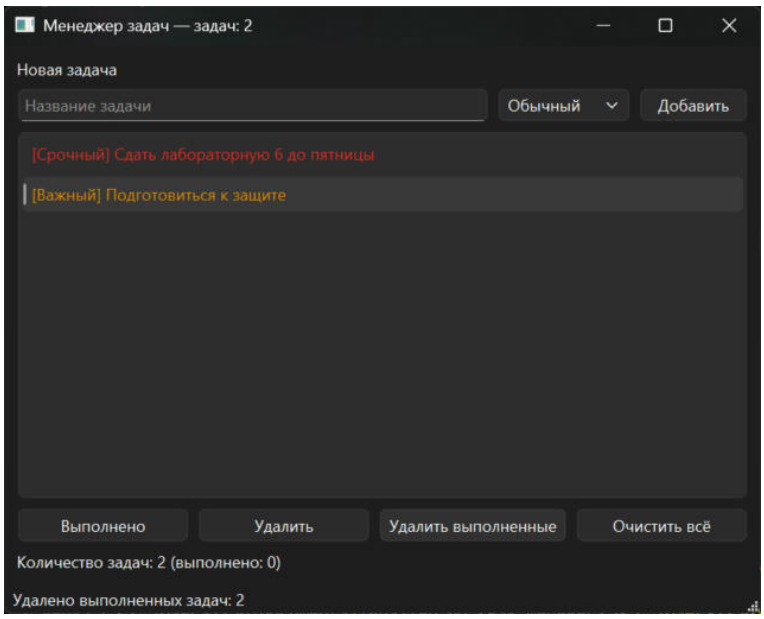


Рисунок 13.2. Удаление выполненных задач

Вывод

В ходе лабораторной работы изучены основы разработки графических интерфейсов на PyQt6. Создано приложение «Менеджер задач»: точка входа main.py создаёт QApplication и главное окно и запускает цикл событий app.exec(), а класс MainWindow на основе QMainWindow описывает окно. В отличие от консольной программы, приложение после запуска ждёт действий пользователя и выполняет код в ответ на события.

Использованы виджеты QLabel, QLineEdit, QPushButton, QListWidget и QMessageBox. Интерфейс собран из вложенных QVBoxLayout и QHBoxLayout на центральном виджете, без абсолютных координат, поэтому при растягивании окна элементы сами меняют размеры.

Действия пользователя обрабатываются через сигналы и слоты: clicked кнопок и returnPressed поля ввода связаны с методами add_task(), delete_task() и clear_tasks(). Некорректные действия (пустой ввод, удаление без выбора, очистка пустого списка) не ломают программу, а дают понятные сообщения; удаление и очистка требуют подтверждения. Метод update_counter() держит счётчик в соответствии со списком.

Код окна разделён на init_ui(), connect_signals() и обработчики, PyQt6 установлен в виртуальное окружение .venv и зафиксирован в requirements.txt, проект находится в Git. Самостоятельно добавлены приоритеты, отметка и удаление выполненных задач, счётчик выполненных, заголовок окна с числом задач и строка состояния — все они встроены в существующие обработчики и не нарушили прежнюю работу.